

Study Companion — Full System Documentation & Technical Manual

Complete Architecture, Project Working, File Breakdown, Data Schemas & Viva Q&A;

1. Executive Overview & End-to-End System Working

Study Companion is an advanced, AI-powered academic workspace designed for university students, educators, and self-learners. It transforms unorganized study materials (PDF textbooks, Word question banks, plain text notes) into structured, unit-classified concept graphs, prerequisite pathways, active-recall quizzes, interactive logic flow simulators, and printable PDF study syllabus guides.

End-to-End User Workflow & System Operation:

- 1. User Registration & Authentication:** Students sign up or log in. Passwords are hashed with bcrypt (passlib) and stateless JWT bearer tokens are issued to authenticate subsequent frontend API requests.
- 2. Subject-Isolated Workspaces:** Users create dedicated subject workspaces (e.g., *Operating Systems*, *Computer Networks*, *Machine Learning*). Documents, concept graphs, chat context, and revision guides remain 100% isolated inside each workspace.
- 3. Document Processing & Portion Feeding:** Documents (.pdf, .docx, .txt) are uploaded or fed directly via the portion text editor. Backend parsers (PyPDF, pdfplumber, python-docx) extract raw text into the database ExtractedText table.
- 4. Multi-Unit Chunking & Syllabus Scanning:** The extraction engine uses regex to split multi-unit files (e.g. *Unit 1..5*, *UNIT-I..V*) into unit chunks. Gemini AI processes each chunk to extract 100% of concepts and questions without hitting output token limits.
- 5. Directed Acyclic Graph (DAG) & Mastery Gating:** Extracted concepts form nodes with prerequisite dependencies. A Depth-First Search (DFS) cycle-resolution algorithm breaks circular dependencies to ensure a clean DAG. Nodes transition from **LOCKED** → **UNLOCKED** → **MASTERED**.
- 6. Mastery Testing (MCQ & Active Recall Explain-Back):** Students unlock concepts by completing 3-question MCQ Quizzes (≥66% score) or writing Explain-Back active recall descriptions scored automatically by Gemini AI (0-100 score).
- 7. Interactive Logic Flow Process Simulator:** Complex algorithms (e.g., Banker's Algorithm, Page Replacement) are visualized as step-by-step flowcharts with playback controls (Play, Pause, Step Forward/Back, Speed).
- 8. Concept Syllabus PDF Report Export:** Generates a formatted PDF report with executive progress statistics, mastered concept breakdown, and unit-by-unit solved study guides.

2. Complete Directory Hierarchy

```
mini-project/
├── backend/
│   ├── app/
│   │   ├── models/           # SQLAlchemy ORM Data Schemas (User, Workspace, Node, Chat...)
│   │   ├── routes/          # FastAPI Endpoint Controllers (Auth, Graph, Content, Chat...)
│   │   ├── schemas/         # Pydantic Request/Response Validation Models
│   │   ├── services/        # Core Business Logic & AI Integrations (Graph, Gemini, Content...)
│   │   ├── utils/           # Security Hashing & Local Storage Utilities
│   │   ├── config.py         # App Environment Variables & Directory Paths
│   │   ├── database.py       # PostgreSQL Engine Setup & Migration Utilities
│   │   ├── main.py           # FastAPI Application Entry Point & Lifespan Handler
│   │   ├── uploads/         # Permanent Uploaded Files Directory
│   │   └── requirements.txt  # Python Dependencies Specification
│   ├── frontend/
│   │   ├── src/
│   │   │   ├── components/  # React Visual UI Components (Roadmap, Chat, LogicFlow...)
│   │   │   ├── styles/      # CSS Design System (Theme Variables & Responsive Layouts)
│   │   │   ├── api.js        # Central Axios API Client & Authentication Layer
│   │   │   ├── App.jsx       # Root Layout & State Manager (Workspace / Profile / Auth)
│   │   │   ├── main.jsx      # React DOM Application Mount Point
│   │   │   └── package.json  # Node.js Dependencies & Build Scripts
│   │   ├── README.md        # Project Overview & Setup Manual
│   │   ├── project-architecture.md # Deep-Dive System Architecture Specification
│   │   └── generate_pdf.py    # ReportLab PDF Documentation Synthesis Script
```

3. Complete File-by-File Breakdown & Responsibilities

Below is a exhaustive description of every file across the backend, frontend, and root system:

- **backend/app/main.py:** FastAPI app entry point. Sets up CORS middleware, lifespan DB initialization, router mounts (/auth, /workspaces, /documents, /content, /chat, /roadmap, /graph), health check endpoints, and static file serving for /uploads.
- **backend/app/config.py:** Central configuration loader using pydantic-settings. Manages DATABASE_URL, JWT_SECRET_KEY, GEMINI_API_KEY, upload directories, and file size/type constraints.
- **backend/app/database.py:** Database connection manager. Configures SQLAlchemy engine, session maker (get_db dependency), Base declarative class, and safe database schema auto-migrations.
- **backend/app/models/user.py:** SQLAlchemy ORM model for Users table (id, name, email, hashed_password, created_at).
- **backend/app/models/workspace.py:** SQLAlchemy ORM model for Workspaces table (id, user_id, name, created_at) with cascade relationships to documents, graph nodes, content, and chat.
- **backend/app/models/document.py:** SQLAlchemy ORM model for Documents table (id, workspace_id, filename, file_path, file_type, file_size, uploaded_at).
- **backend/app/models/extracted_text.py:** SQLAlchemy ORM model for ExtractedText table (id, document_id, raw_text, created_at).
- **backend/app/models/concept_graph.py:** SQLAlchemy ORM models for ConceptNode (title, unit, summary, sub_points, answer_cache), ConceptEdge (prerequisite links), and NodeMastery (LOCKED/UNLOCKED/MASTERED states).
- **backend/app/models/generated_content.py:** SQLAlchemy ORM model for GeneratedContent table storing AI outputs (REVISION, EXAM, QUIZ, LOGIC_FLOW) with titles and metadata.
- **backend/app/models/chat_history.py:** SQLAlchemy ORM model for ChatHistory table (id, workspace_id, user_message, ai_response, node_title, node_summary, timestamp).
- **backend/app/routes/auth.py:** Endpoints for User Registration (/auth/register), User Login (/auth/login returning JWT), and Current Profile (/auth/me).
- **backend/app/routes/workspaces.py:** Endpoints for CRUD operations on Workspaces (/workspaces, GET, POST, DELETE).
- **backend/app/routes/documents.py:** Endpoints for Document Upload (/upload), Document Deletion, Text Fetch, and Portion Text Feeding.

- **backend/app/routes/graph.py:** Endpoints for Graph Generation (/graph/generate), Graph Retrieval (/graph), Node Answer Fetch (/node/answer), PDF Export (/pdf/export), Clear Feed (/feed), and Mastery Attempt (/attempt).
- **backend/app/routes/content.py:** Endpoints for generating structured Markdown Revision Guides, Solved Exam Guides, MCQ Quizzes, and Logic Flow JSON.
- **backend/app/routes/chat.py:** Endpoints for Workspace Chat (/chat/send) and Chat History Retrieval (/chat/history).
- **backend/app/services/graph_service.py:** Core graph algorithms: Multi-unit chunk extraction, clean_unit_name regex normalization, DFS cycle detection/stripping (detect_and_resolve_cycles), mastery state evaluation, explain-back AI grading, and reference answer caching.
- **backend/app/services/content_service.py:** Prompt engineering and AI generation service for Revision Guides, Solved Exam Question & Answer Guides (with Examiner Coaching Boxes), MCQ Quizzes, and Logic Flow step JSON.
- **backend/app/services/chat_service.py:** Workspace-bounded chat service. Merges document context with active concept node focus to generate contextually accurate AI answers.
- **backend/app/services/gemini_service.py:** Google Gemini API wrapper (gemini-2.5-flash). Handles prompt execution, system instructions, and robust JSON auto-repair parsing (parse_json_from_ai).
- **backend/app/services/document_service.py:** File upload processing, extension validation, file storage on disk, and triggering PyPDF/docx text extraction.
- **backend/app/services/pdf_service.py:** PyPDF and pdfplumber document parser for page-by-page text extraction.
- **backend/app/services/docx_service.py:** python-docx document parser for extracting text from Word documents.
- **backend/app/utis/security.py:** Security helper functions: bcrypt password hashing (hash_password, verify_password), JWT creation (create_access_token), and token payload decoding.
- **frontend/src/App.jsx:** Root React component. Manages auth session persistence, dark/light theme state, workspace routing, tab manager lifecycle, and chat state.
- **frontend/src/api.js:** Axios API client wrapper. Attaches Authorization: Bearer JWT header to all backend requests and handles unified error handling.
- **frontend/src/components/AnimatedRoadmap.jsx:** Right-hand sidebar visualizer. Handles portion text feeding, format selection, unit tabs display, node lock status rendering, and workspace concept progress bar.
- **frontend/src/components/TabManager.jsx:** Center panel with browser-style tabs. Pins Chat Copilot tab, renders opened document previews, node reference answers, revision content, and interactive quizzes.
- **frontend/src/components/ConceptGraphPanel.jsx:** Interactive canvas visualizer rendering concept nodes, topological dependency edges, unit tab filters, and node mastery status badges.
- **frontend/src/components/LogicFlowPanel.jsx:** Step-by-step flowchart simulator with interactive playback controls (Play, Pause, Step Forward/Back, Speed selector) and step details.
- **frontend/src/components/WorkspaceAnalytics.jsx:** Student profile dashboard displaying overall mastery progress, study stats, workspace management list, and focus timer overview.
- **frontend/src/components/FocusTimer.jsx:** Header Pomodoro focus timer tracking active study time per session with Start/Pause controls.
- **frontend/src/components/QuizPanel.jsx:** Interactive 3-question MCQ quiz modal with immediate grading, answer explanations, and mastery trigger.
- **frontend/src/components/AuthForm.jsx:** Clean user authentication card rendering Login and Register forms with toggleable inputs.
- **frontend/src/components/WorkspaceSidebar.jsx:** Left-hand sidebar displaying current workspace status, uploaded document list, file upload trigger, and back to workspaces navigation.
- **frontend/src/components/MarkdownRenderer.jsx:** Rich Markdown parser component supporting syntax highlighted code blocks, tables, callout boxes, LaTeX equations, and copy buttons.

4. Technical Architecture & Core Algorithms

Database Entity Relationship (ER) Schema Overview:

Table Name	Key Columns & Types	Relationships & Foreign Keys
users	id (UUID), email (String), hashed_password	Has Many <i>workspaces</i>
workspaces	id (UUID), user_id (FK), name	Belongs to <i>user</i> . Has Many <i>documents</i> , <i>concept_nodes</i> , <i>chat_history</i>
documents	id (UUID), workspace_id (FK), filename, file_path	Has One <i>extracted_text</i>
extracted_text	id (UUID), document_id (FK), raw_text (Text)	Belongs to <i>document</i>
concept_nodes	id (UUID), workspace_id (FK), title, unit, summary, answer_cache	Has Many <i>concept_edges (prerequisites)</i> , <i>node_mastery</i>
node_mastery	id (UUID), node_id (FK), status (LOCKED/UNLOCKED/MASTERED), score	Belongs to <i>concept_node</i>
chat_history	id (UUID), workspace_id (FK), user_message, ai_response	Belongs to <i>workspace</i>

Core Algorithmic Deep Dives:

- **DFS Cycle Resolution Algorithm:** When AI generates prerequisite edges, circular dependencies (e.g. Node A → Node B → Node C → Node A) could break topological sorting. The system runs a Depth-First Search (DFS) recursion tracking active recursion stacks. When a back-edge pointing to an active parent node is detected, that back-edge is stripped out, guaranteeing a pure Directed Acyclic Graph (DAG).
- **Multi-Unit Scanning Engine:** Text extracted from documents is passed through regex patterns matching unit markers (Unit 1 to Unit 5, UNIT-I to UNIT-V, *Module 1..5*). The text is chunked into unit segments and fed into Gemini AI in separate parallel passes. This prevents token truncation and guarantees 100% question coverage across all units.
- **AI JSON Auto-Repair Engine (parse_json_from_ai):** AI outputs often contain unescaped quotes, Markdown code block fences, or truncated brackets. The repair engine strips Markdown backticks, sets strict=False for control characters, applies regex to find valid JSON blocks, and programmatically appends closing braces/brackets if the response was truncated.
- **Answer Caching:** To minimize LLM API cost and reduce response latency from 3s to 10ms, generated node reference answers are cached directly in the ConceptNode.answer_cache column in PostgreSQL. Subsequent node clicks load instantly from DB.

5. Complete REST API Endpoint Catalog

Method	Endpoint Path	Auth Required	Description & Purpose
POST	/auth/register	No	Registers a new user account with hashed password.
POST	/auth/login	No	Authenticates credentials and returns JWT access token.
GET	/auth/me	Yes (JWT)	Returns current authenticated user profile details.
GET	/workspaces	Yes (JWT)	Lists all workspaces owned by the authenticated user.
POST	/workspaces	Yes (JWT)	Creates a new subject workspace.
DELETE	/workspaces/{id}	Yes (JWT)	Deletes a workspace and all associated documents & graphs.
POST	/workspaces/{id}/upload	Yes (JWT)	Uploads a PDF, DOCX, or TXT file to the workspace.
POST	/workspaces/{id}/graph/generate	Yes (JWT)	Runs multi-pass LLM concept extraction & creates concept graph.
GET	/workspaces/{id}/graph	Yes (JWT)	Fetches concept nodes, dependency edges, unit list, and mastery states.
GET	/workspaces/{id}/node/{node_id}/answer	Yes (JWT)	Fetches cached or AI-generated reference study answer for a concept.
POST	/workspaces/{id}/attempt	Yes (JWT)	Submits an MCQ quiz attempt or AI Explain-Back response.
GET	/workspaces/{id}/pdf/export	Yes (JWT)	Generates and downloads the printable ReportLab PDF syllabus report.
POST	/workspaces/{id}/chat/send	Yes (JWT)	Sends a chat message to the workspace-aware AI copilot.

6. Viva & Technical Project Review Q&A; Cheat Sheet

Comprehensive questions and answers commonly asked during academic viva evaluations and code reviews:

Q1: What is the primary problem Study Companion solves?

Study Companion addresses fragmented study materials by isolating courses into dedicated workspaces, automatically extracting unit-wise concept syllabi, mapping prerequisite learning paths using Directed Acyclic Graphs (DAGs), enforcing active-recall mastery, and providing workspace-grounded AI assistance without context cross-contamination.

Q2: Why did you select FastAPI over Flask or Django?

FastAPI is an asynchronous ASGI web framework built on Python 3.11 standards. It offers non-blocking async I/O performance on par with Node.js and Go, enforces strict request/response data validation via Pydantic, and automatically generates interactive OpenAPI documentation (/docs).

Q3: How does JWT authentication secure API endpoints?

During authentication in `auth_service.py`, the backend signs a JSON Web Token containing the user ID using HS256 algorithm and a secret key. The frontend includes this token in the 'Authorization: Bearer ' header. Routes enforce authentication using FastAPI's `Depends(get_current_user)` dependency.

Q4: How is subject isolation enforced across workspaces?

Every database table (Documents, ConceptNodes, GeneratedContent, ChatHistory) enforces a mandatory `workspace_id` foreign key constraint. All query filters strictly condition on the active `workspace_id`, ensuring the AI model context window is populated exclusively with documents from the active subject.

Q5: How do you handle circular dependencies in AI-generated concept graphs?

In `graph_service.py`, we implement a Depth-First Search (DFS) cycle resolution algorithm (`detect_and_resolve_cycles`). As edges are traversed, nodes currently in the recursion stack are tracked. If a back-edge is discovered pointing to an ancestor node, the edge is deleted, ensuring a valid Directed Acyclic Graph (DAG).

Q6: How does the system handle multi-unit question banks without missing topics?

We use a regex unit splitter that detects section markers ('Unit 1', 'UNIT-I', 'Module 1'). The text is split into unit chunks, and Gemini AI performs multi-pass extraction per unit chunk. The unit tags are normalized using `clean_unit_name` to ensure 100% concept coverage across all units.

Q7: How does active recall ('Explain-Back') evaluation work?

When a student submits an explanation of a concept, Gemini AI evaluates the explanation against the concept's summary. The AI returns a score (0-100) and actionable coaching feedback. If the score is $\geq 70\%$, the concept state updates to MASTERED and downstream dependent nodes unlock automatically.

Q8: What mechanism prevents high latency on repeated concept reference answer clicks?

We implement Answer Caching. Upon the first request for a concept's reference answer, Gemini AI generates the detailed Markdown response which is written to `ConceptNode.answer_cache` in PostgreSQL. Subsequent clicks load instantly from the database without invoking the Gemini API.

Q9: How are unescaped control characters or truncated JSON from AI handled?

Through our `parse_json_from_ai` utility function in `gemini_service.py`. It strips Markdown fences (``json), parses text with `strict=False` to tolerate raw line breaks, uses regular expressions to locate valid JSON structure boundaries, and auto-balances missing closing brackets if truncated.

Q10: How is the Concept Syllabus PDF Report synthesized programmatically?

We utilize ReportLab's flowable document architecture (`SimpleDocTemplate`, `Paragraph`, `Table`, `HRFlowable`). The backend builds structured visual elements containing executive statistics, mastery status lists, and unit-grouped reference study guides, streaming the resulting binary PDF directly to the client browser.